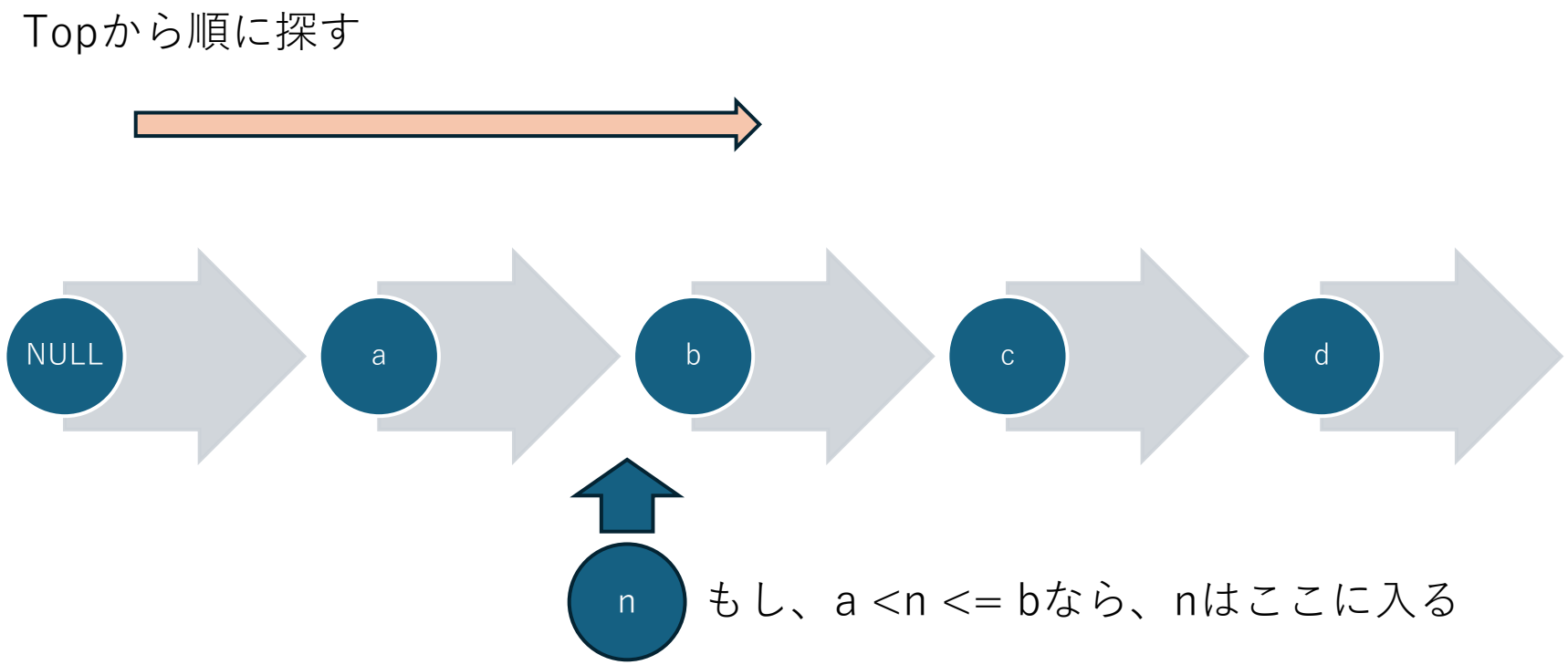
46-50

問46: 整列した単方向リスト

- 目的: 昇順に整列した単方向リストに整列を保って要素を追加
- 変数

  prev: ひとつ前の要素, curr: 現在の要素, work: 作業用の要素
- ポイント
  - topを未定義で初期化しているが、これは最初はリストが空だから
  - whileループは, currを次に進めている→追加要素を入れる場所を探している→currがnumと同じか小さければ、prevとcurrの間に挿入
  - if (currがtopと等しい) は、リストが空ということ

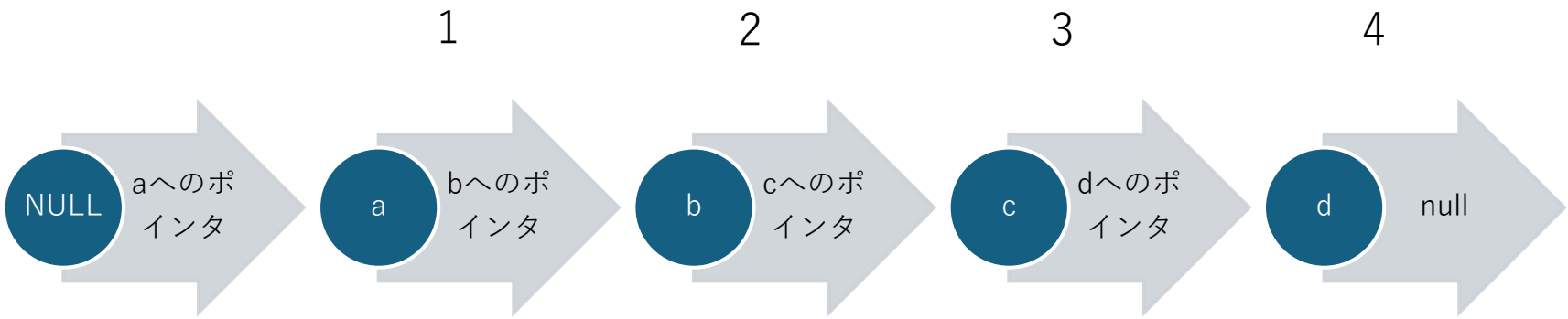
図 46-1



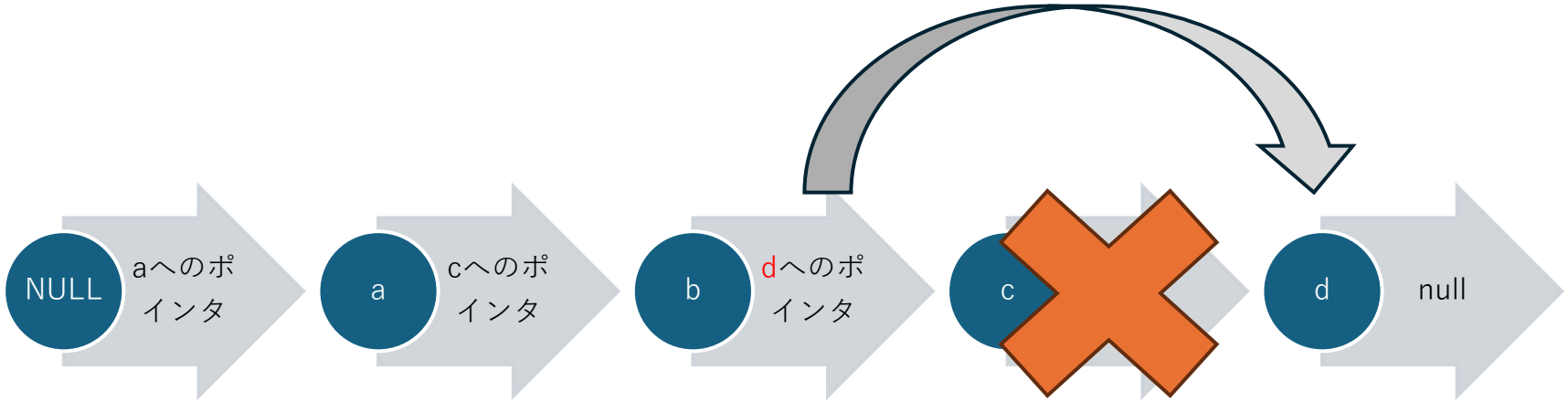
問47: 単方向リスト・要素の削除

- 目的: 要素の削除
- 変数
  - cur: 現在のノード, prev: ひとつ前のノード, cnt: カウンタ
- ポイント
  - **変更は削除する前の要素で発生**（削除は、前の要素のポインタ部分を削除対象要素から、その次の要素へ書き換えること）
  - Top ← cur.next は先頭の要素を削除すること
  - for文で、先頭からorderだけリストを進めている

図 47-1



bを削除するには、aの「bへのポインタ」を「cへのポインタ」に書き換える



問48 二分探索木への挿入

二分探索木の「鉄の掟」

二分探索木では、すべてのノードにおいて以下のルールが守られています。

- **左の子**のノード（およびその先にあるすべての値）は、**親よりも小さい**
- **右の子**のノード（およびその先にあるすべての値）は、**親よりも大きい**

このルールが再帰的に適用されているため、どの部分を切り取っても「左 < 親 < 右」という関係が維持されます。

- ポイント
  - qKeyがp(現在のノード)のKeyより小さい場合は左、大きい場合は右に追加する場所を探す

図48-1

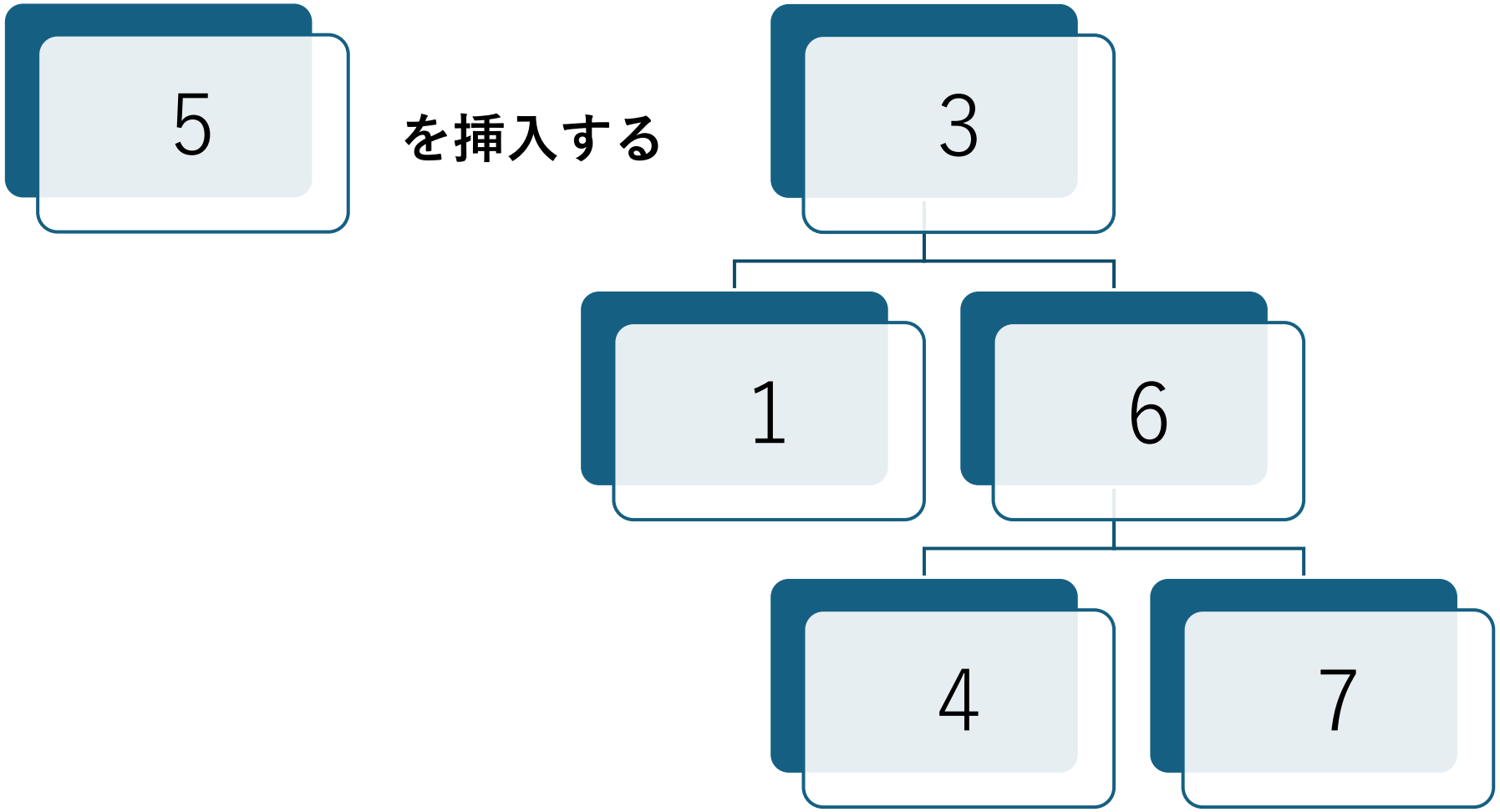


図48-2

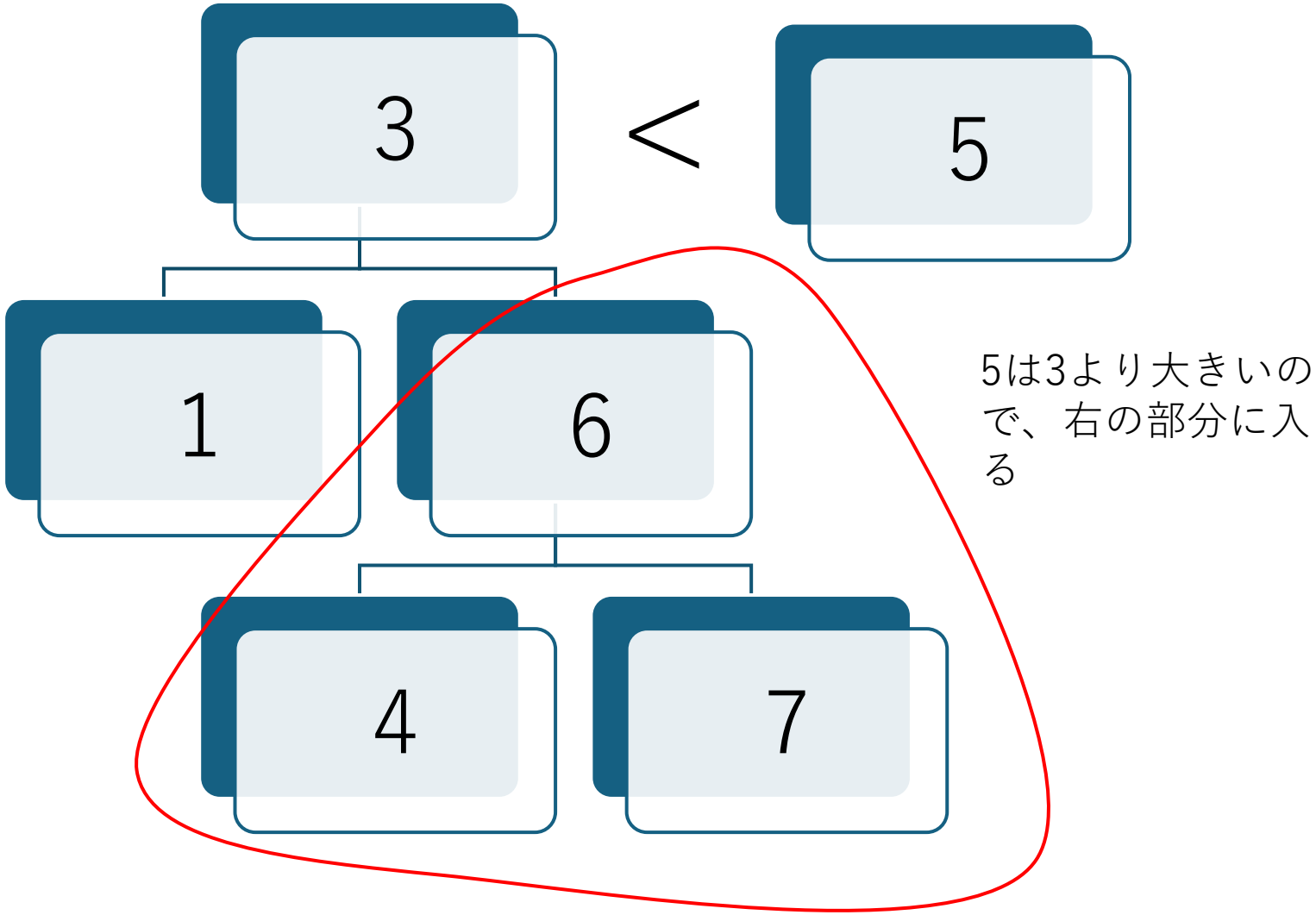


図48-3

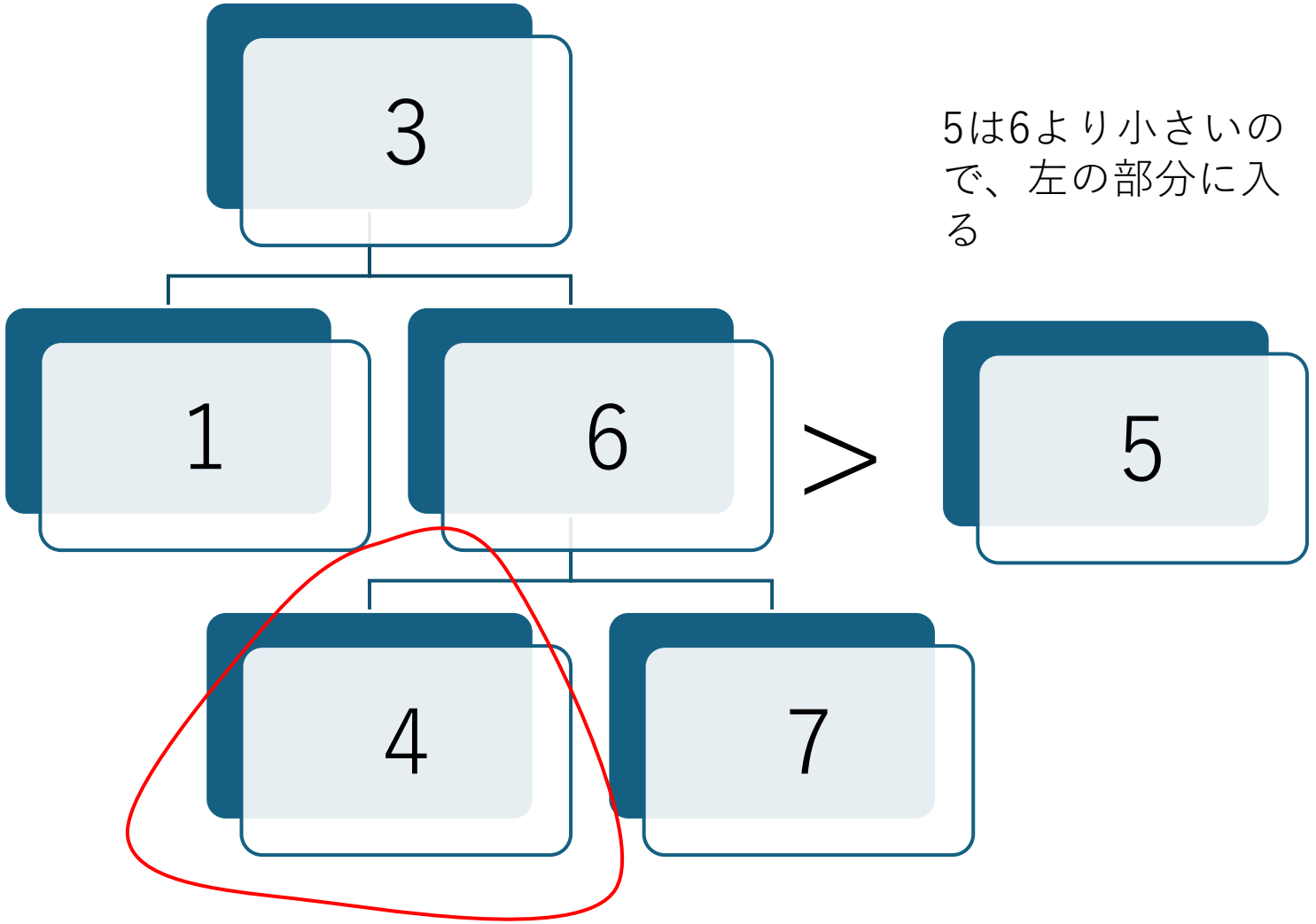


図48-5

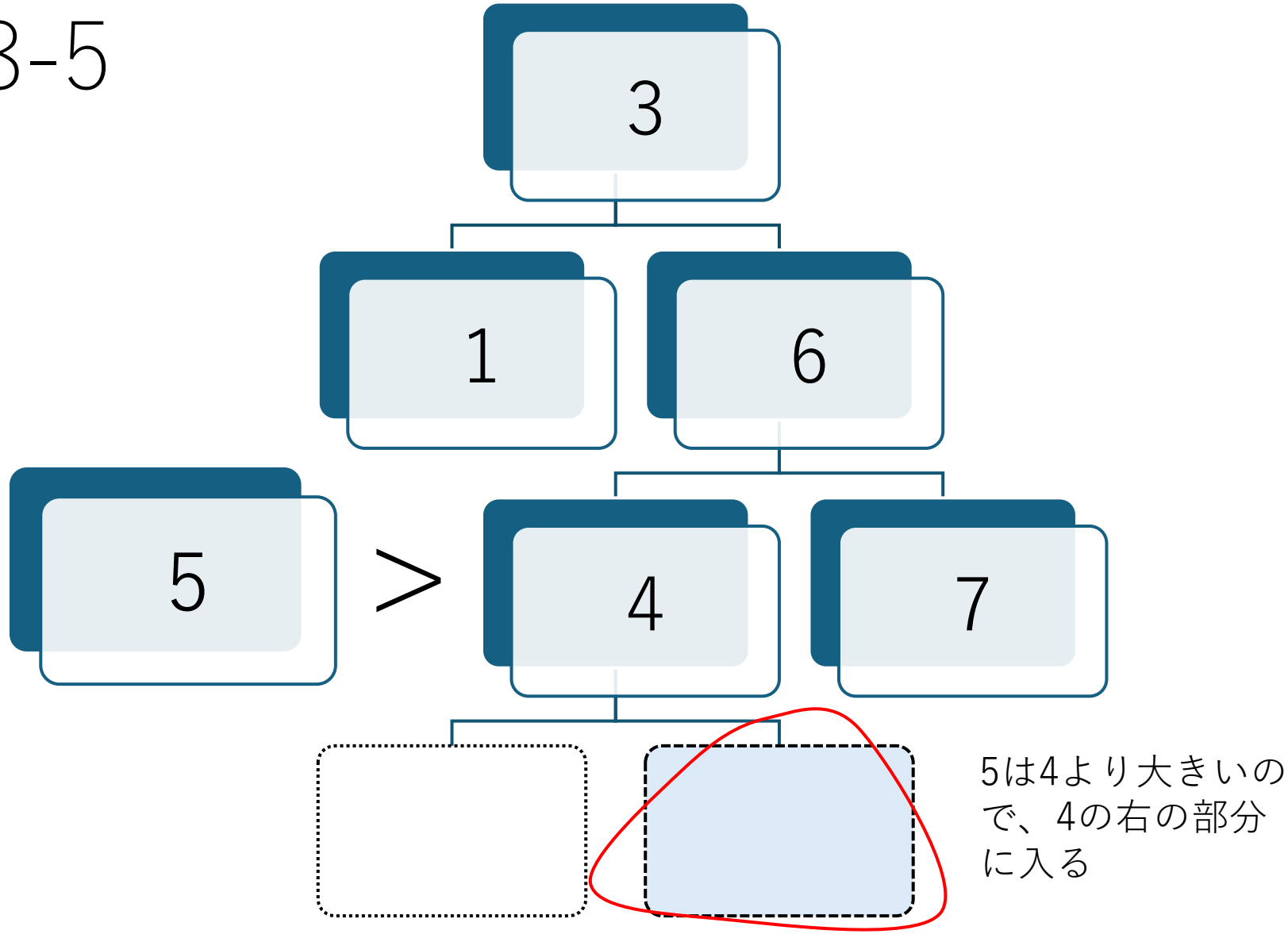
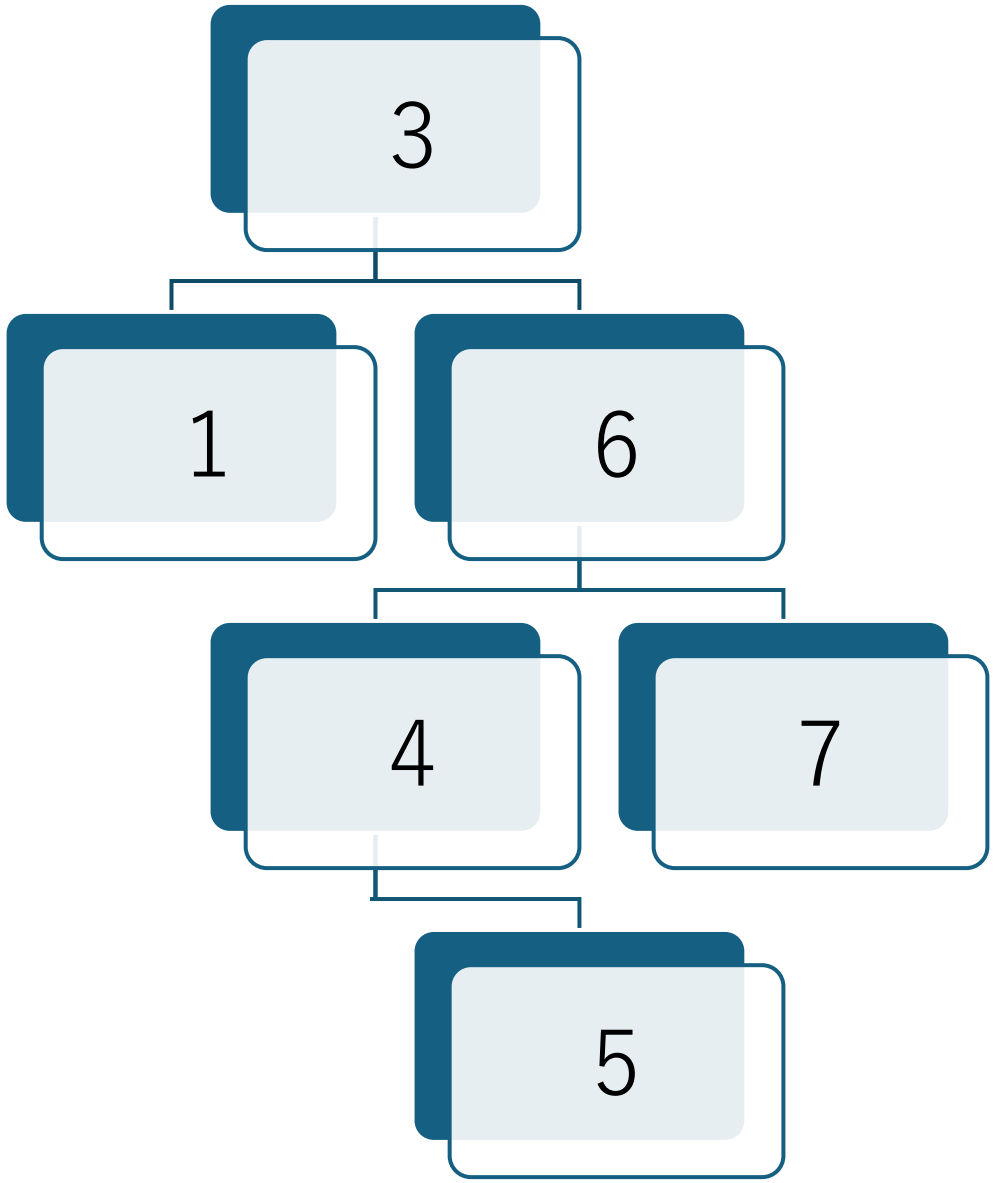


図48-5



問49: 幅優先探索

- 目的: 幅優先探索で全ノードの値を出力
- 変数

  queue: 探索結果を入れる, node: 出力するノード, head: 配列queueの未出力の先頭indx, tail: queueの末尾index
- ポイント
  - whileループはすべてのノードを巡回する
  - 全階層を出力したら、node.left, node.rightとも未定義になる
  - このとき、head はインクリメントされるが、tailはインクリメントされない→ head == tailになる ※ これがループ終了条件
  - 左のノードが右のノードより先に出力されるので、左が先にqueueに入る ※ queueは先入れ先出し

問50: 深さ優先探索

- 目的: 深さ優先探索ですべてのノードを出力
- 変数

  stack: 出力データを格納する, node1: 探索を進める, node2: スタックからpopする, sp: スタックポインタ
- ポイント
  - stackは後入れ先出し
  - aはstackにpush, bはstackからpop
  - 外のループはすべて取り出した後にぬける
  - 内のループは